



CS683: Advanced Computer Architecture-2024

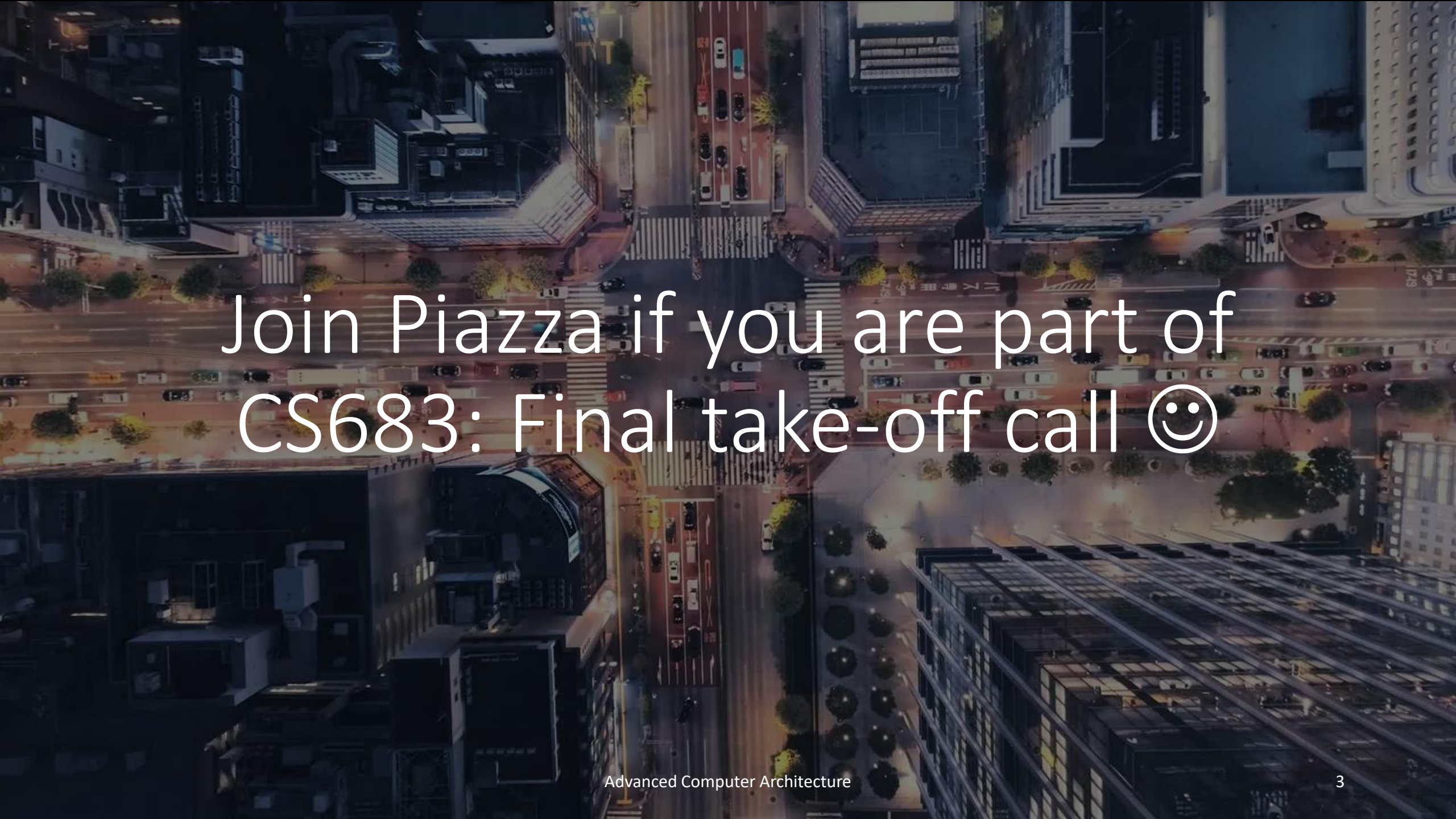
L3-Beyond Cache and beyond

<https://www.cse.iitb.ac.in/~biswa/courses.html>

<https://www.cse.iitb.ac.in/~biswa/>



Phones (smart/non-smart)
on silence plz, Thanks

An aerial night view of a city intersection. The scene is illuminated by streetlights and building lights, creating a warm glow. Several cars are visible on the roads, and the surrounding buildings are tall and modern. The text "Join Piazza if you are part of CS683: Final take-off call 😊" is overlaid in the center in a white, sans-serif font.

Join Piazza if you are part of
CS683: Final take-off call 😊

Summary of Lecture-2



Confused?

Cache Miss Analysis

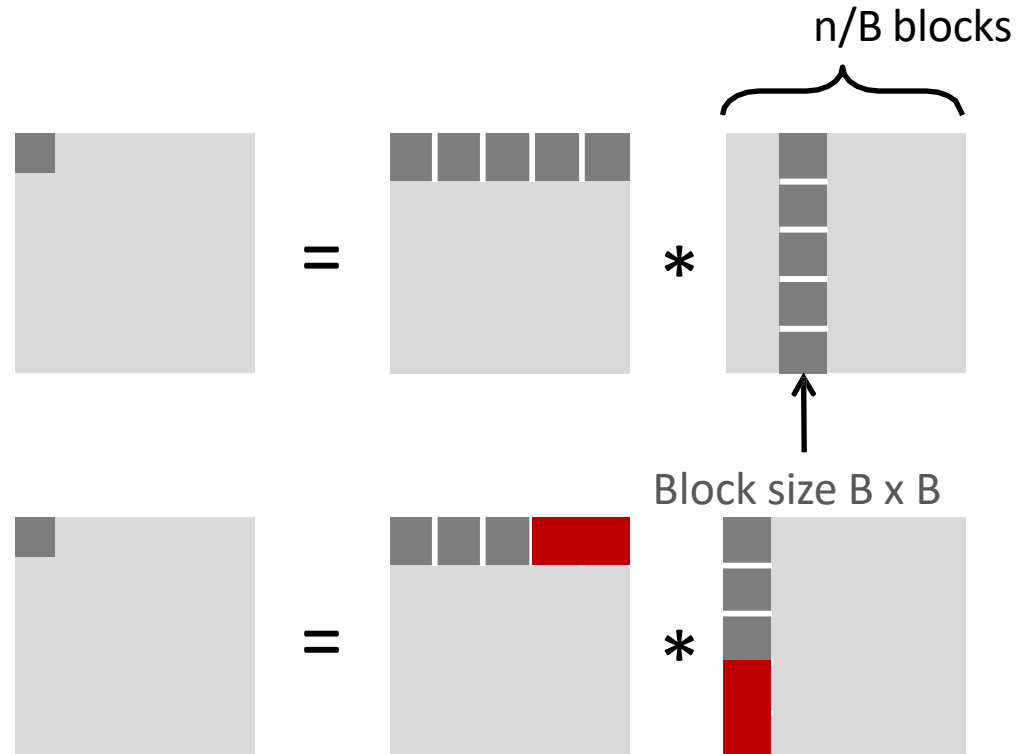
- Assume:
 - Cache block = 8 doubles
 - Cache size $\ll n$ (much smaller than n)
 - Three blocks  fit into cache, i.e., $3B^2 < C$

- First (block) iteration:

- $\frac{B^2}{8}$ misses for each block (tile)
- $2 * \frac{n}{B} * \frac{B^2}{8} = \frac{2*nB}{8}$

(ignoring matrix C)

- Afterwards in cache (schematic)



Cache Miss Analysis

- Assume:
 - Cache block = 8 doubles
 - Cache size $\ll n$ (much smaller than n)
 - Three blocks  fit into cache, i.e., $3B^2 < C$

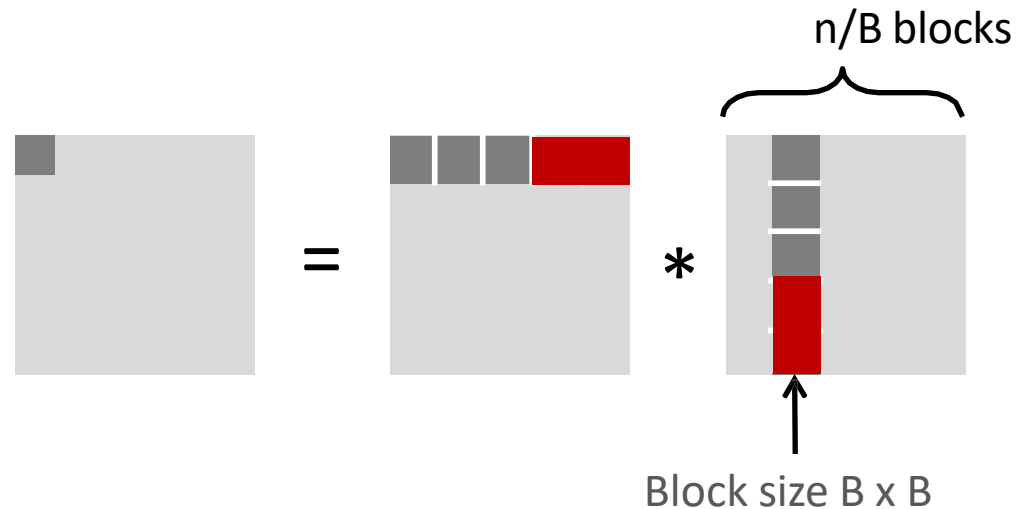
- Second (block) iteration:

- Same as first iteration

- $2 * \frac{n}{B} * \frac{B^2}{8} = \frac{2 * n B}{8}$

- Total misses:

- $\frac{2 * n B}{8} * \left(\frac{n}{B}\right)^2 = \frac{2 * n^3}{8 * B}$



Summary

Without blocking	With blocking
$\frac{9}{8} * n^3$	$\frac{2}{8B} * n^3$

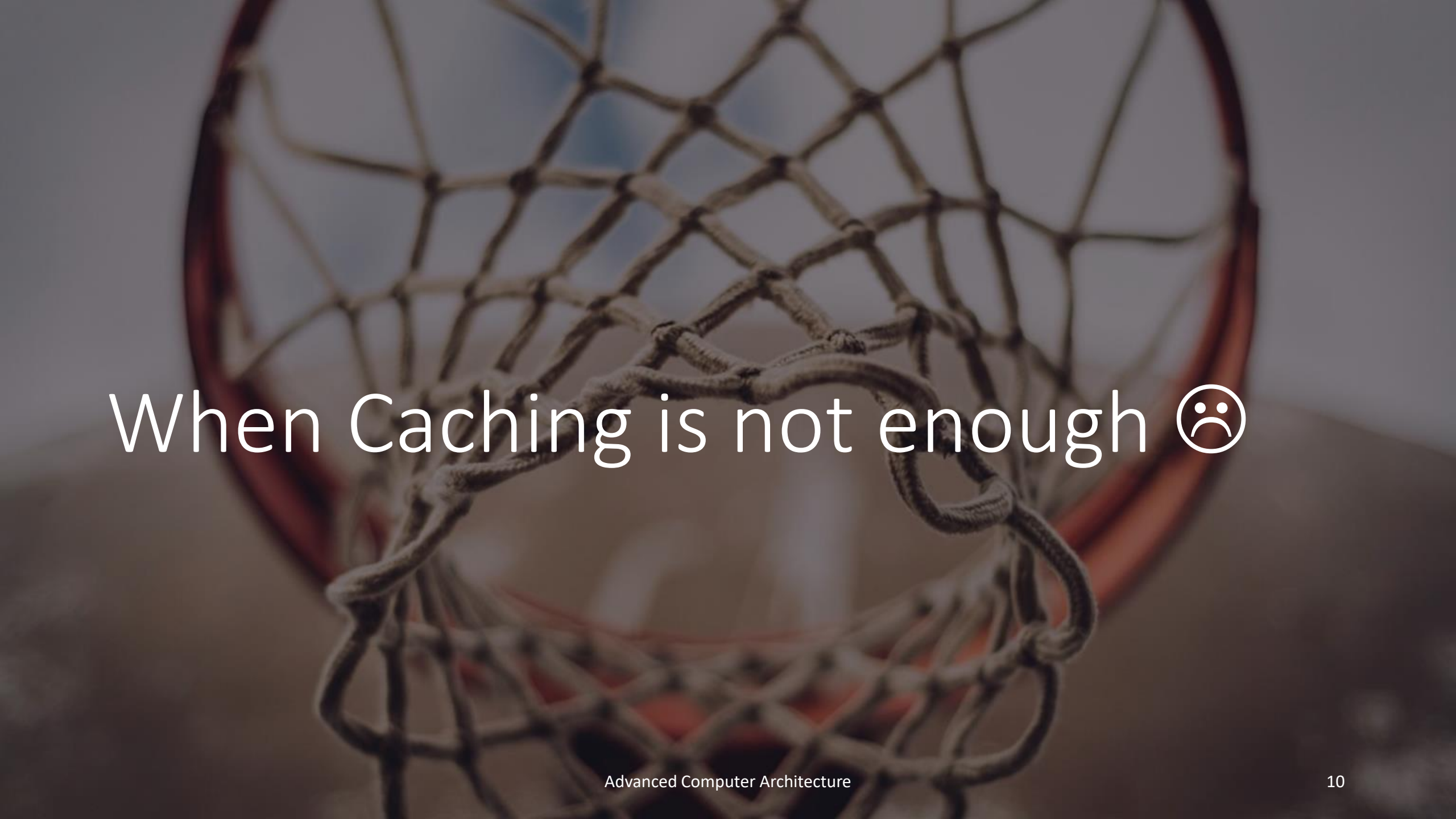
- Find largest possible block size BLK , but limit $3BLK^2 < C$!
- Reason for dramatic difference is that matrix multiplication has inherent temporal locality
 - Input data is $3n^2$, computation is $2n^3$, and every array element is used $O(n)$ times!
 - But the program has to be written properly (choose good variant and BLK)

An aerial photograph of a winding asphalt road that snakes through a dense, snow-covered evergreen forest. The road has white lane markings and a few small vehicles are visible in the distance. The trees are heavily laden with snow, creating a high-contrast, textured landscape. In the top left corner, there is a small orange rectangular graphic element.

Speedup

If $B = 16$, 72X speedup 😊 😊

Why 16?



When Caching is not enough ☹️

Let's do software data prefetching
(Compiler can do it too)

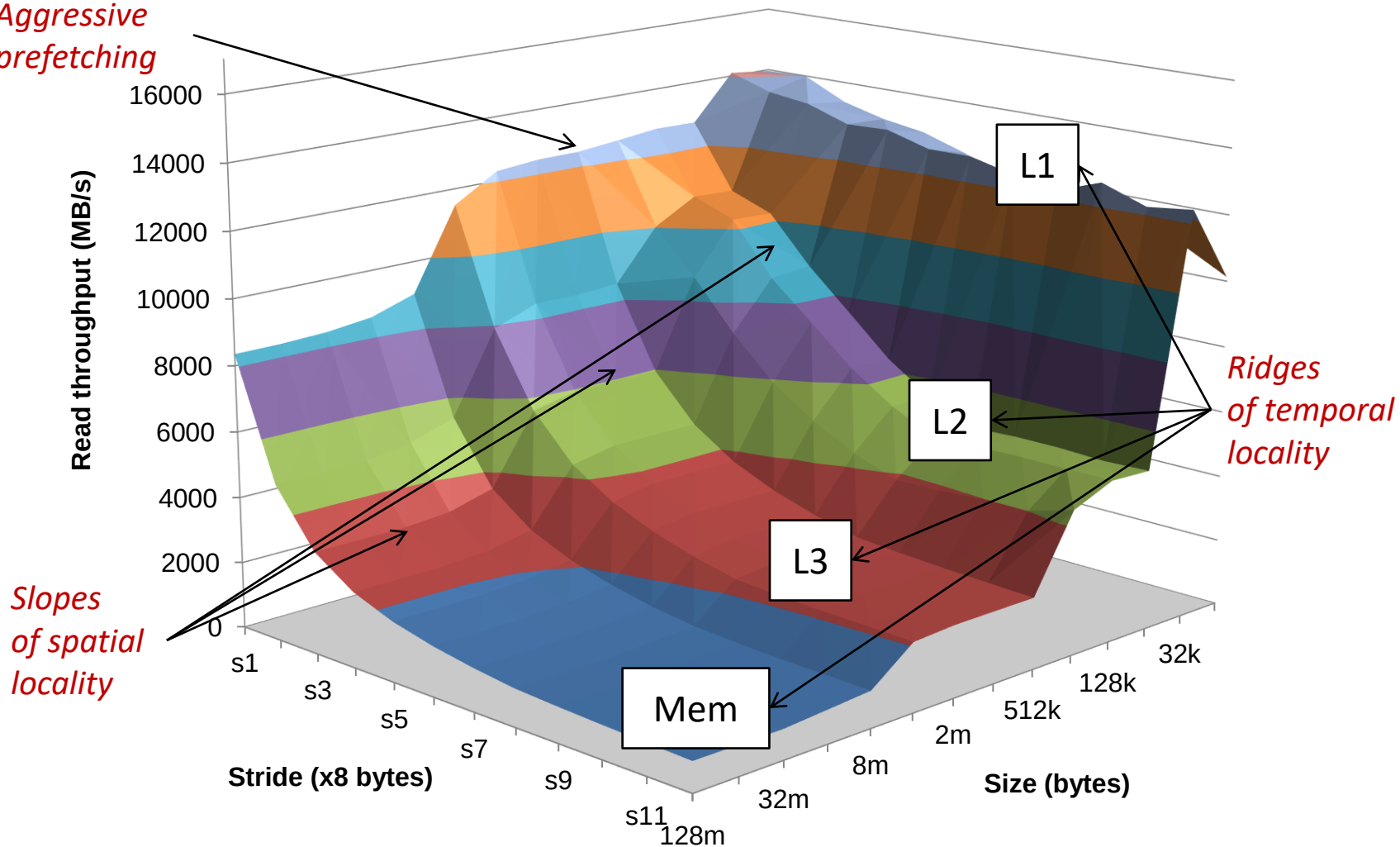
<https://johnnysswlab.com/the-pros-and-cons-of-explicit-software-prefetching/>

Try this in gcc and clang

```
for (size_t i = 0; i < n; i++) {  
    __builtin_prefetch(&a[index[i + 8]]);  
    a[index[i]]++;  
}
```

Memory Mountain

*Aggressive
prefetching*



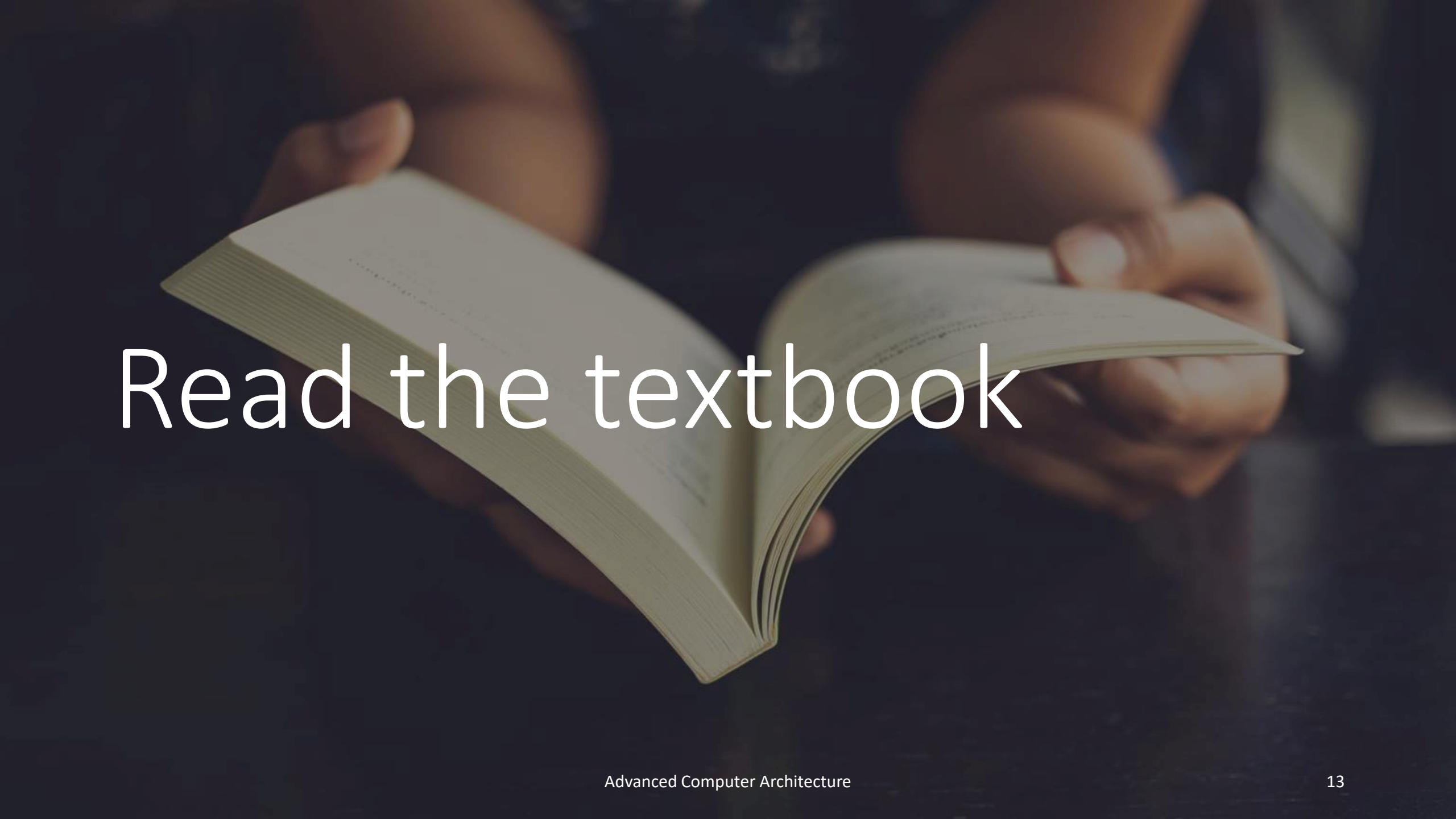
Core i7 Haswell

2.1 GHz
32 KB L1 d-cache

256 KB L2 cache

8 MB L3 cache

64 B block size

A close-up, slightly blurred photograph of a person's hands holding an open textbook. The person is wearing a dark blue shirt. The background is dark and out of focus. The text 'Read the textbook' is overlaid in a large, white, sans-serif font across the center of the image.

Read the textbook



What else ? I still need performance

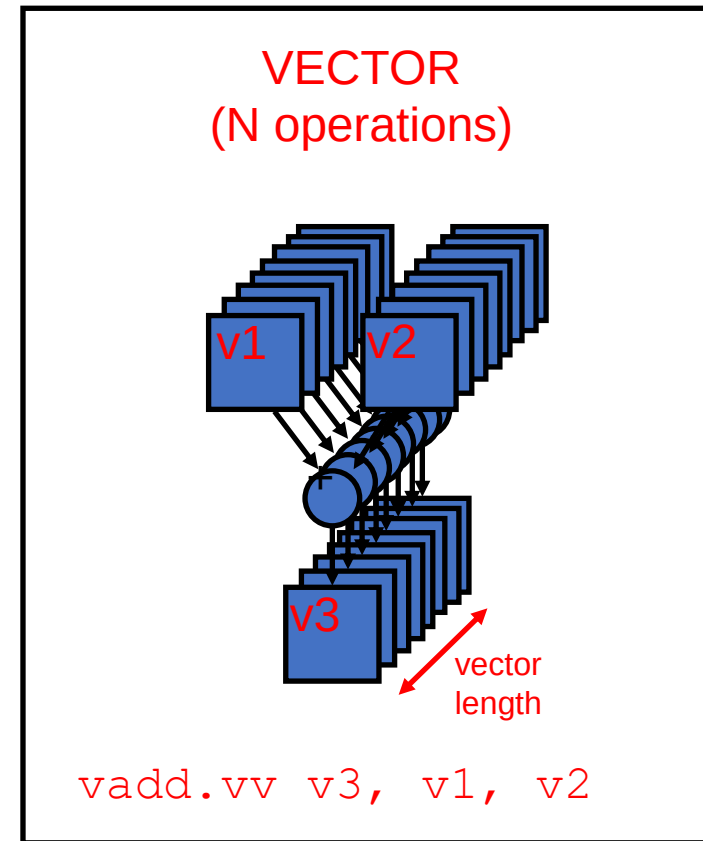
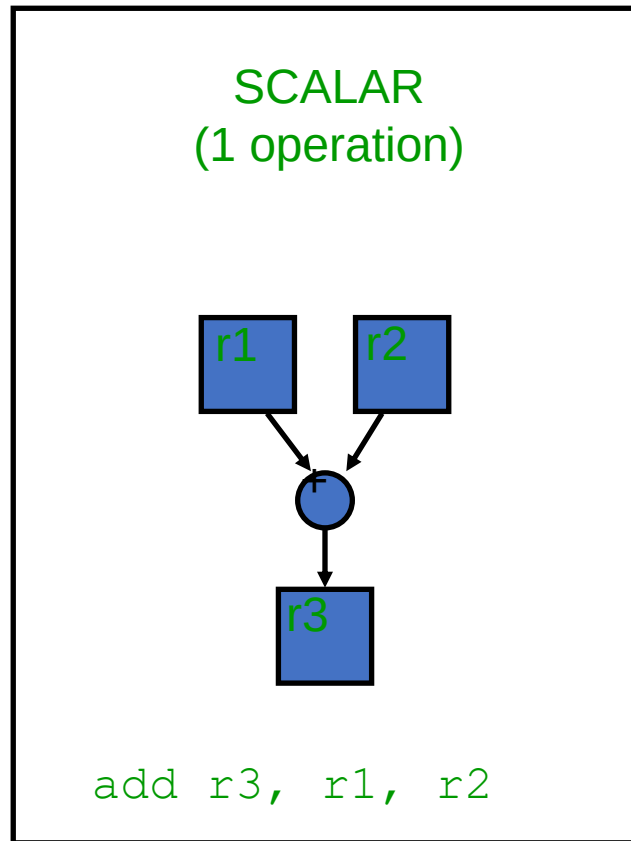
SIMD magic

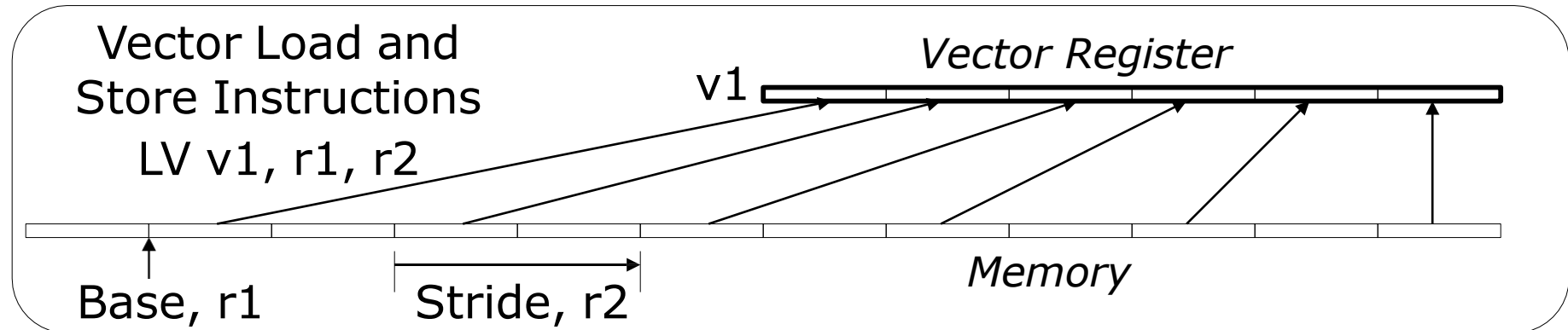
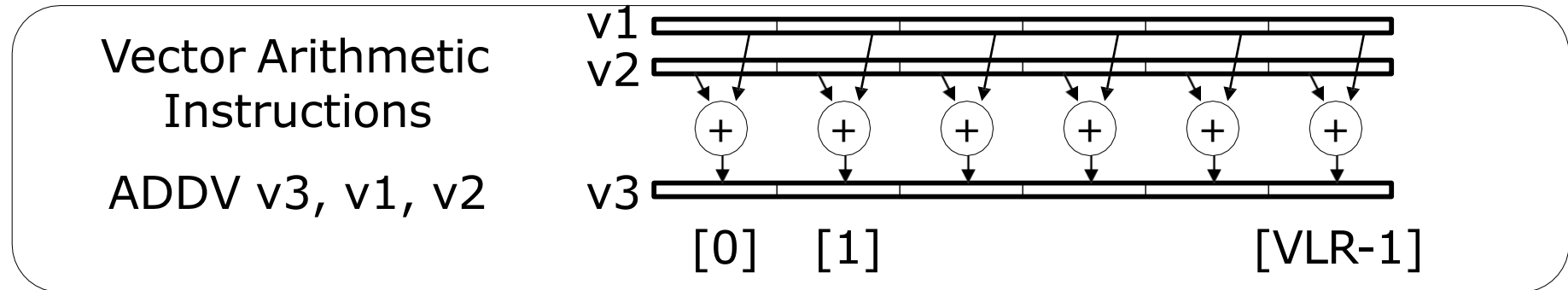
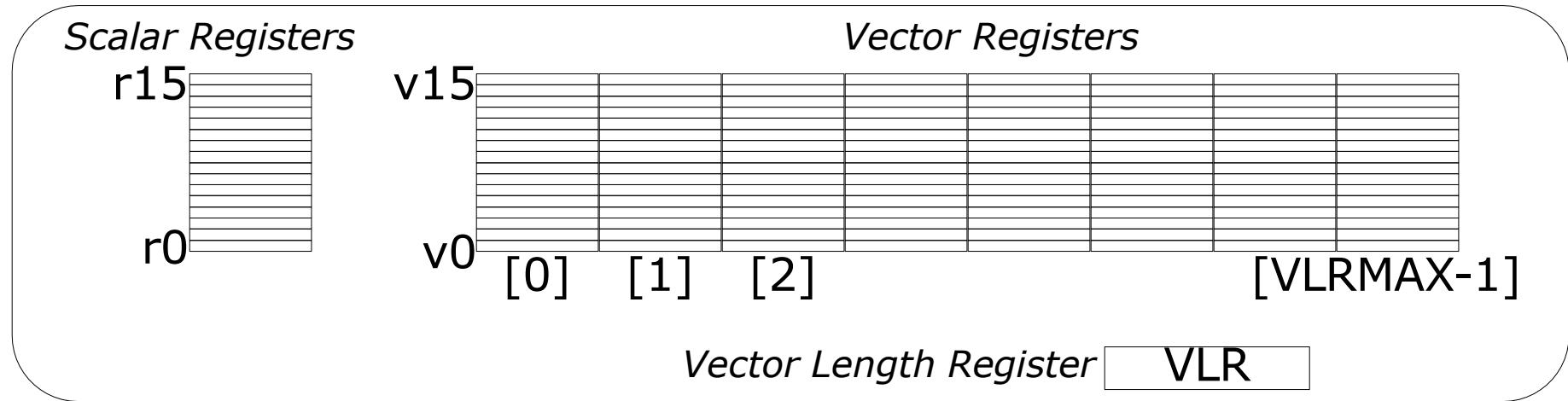
- Single instruction on Multiple Data magic
- SIMD is good for applying identical computations across many data elements – E.g.

```
for(i = 0; i < 100; i++) {  
  A[i] = B[i] + C[i];  
}
```

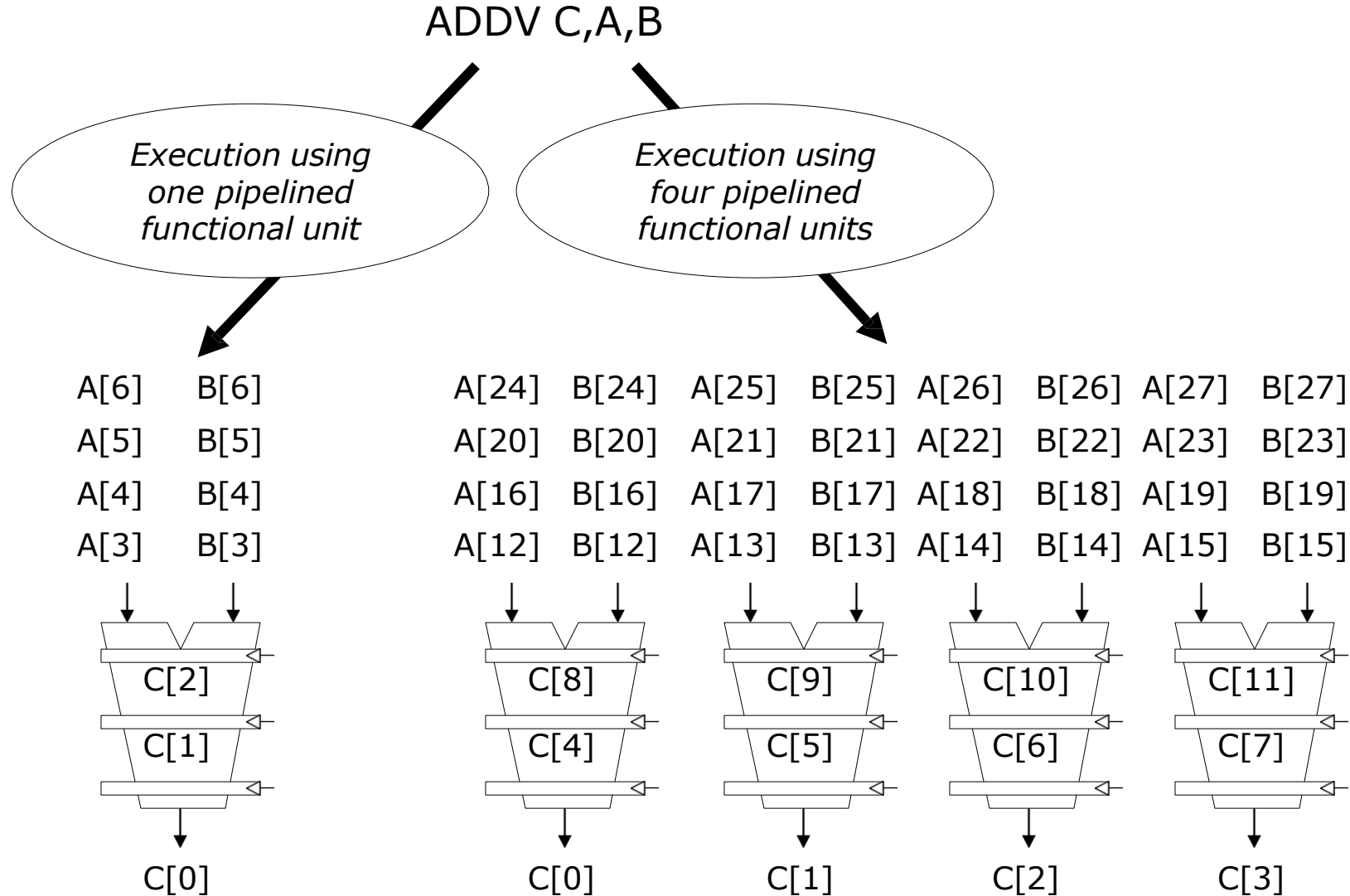
Also “data level parallelism”

The magic (Compiler (programmer) ensures no dependencies)





More Execution Units the Better



Look for

- SSE instructions in x86

[https://en.wikipedia.org/wiki/Streaming SIMD Extensions](https://en.wikipedia.org/wiki/Streaming_SIMD_Extensions)

Refer Textbook H&P

Programming Assignment-1 Heads-up (coming on 13th August)